



Estimating and Differentiating Programming Skills from Unstructured Coding Exercises via Matrix Factorization

Fabian Pfütsch

Ostfalia University of Applied Sciences, Dept. of Computer Science

Germany

7q2s4cwu@duck.com

Frank Höppner

Ostfalia University of Applied Sciences, Dept. of Computer Science

Germany

f.hoeppner@ostfalia.de

Abstract

While *framed* programming exercises (e.g. multiple choice questions or filling blanks in code) are much easier to evaluate, *unstructured* coding exercises allow a more realistic assessment of actual programming skills. We investigate if we can assess programming skills from unstructured coding exercises automatically using a Matrix Factorization (MF) approach. From the *evolution of passed unit-tests over time*, we derive a performance metric per exercise attempt. Different MF models are fitted and we investigate to what extent the learned model parameters express programming skills. The parameters correlate well with manually assessed skills, allow us to distinguish the capabilities regarding varying language constructs, and reveal trends in the skill evolution. The approach provides meaningful insights into programming skills that may be used to improve formative feedback in the future.

CCS Concepts

• **Social and professional topics** → **Computer science education; Student assessment**; • **Computing methodologies** → **Factorization methods**.

Keywords

Learning Progress, Knowledge Tracing, Programming, Coding, Exercises, Matrix Factorization

ACM Reference Format:

Fabian Pfütsch and Frank Höppner. 2024. Estimating and Differentiating Programming Skills from Unstructured Coding Exercises via Matrix Factorization. In *24th Koli Calling International Conference on Computing Education Research (Koli Calling '24)*, November 12–17, 2024, Koli, Finland. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3699538.3699552>

1 Introduction

In this work we consider the assessment of programming skills in introductory programming courses. Learning to program is a challenging task for many students, it is therefore important to support them on all levels of the Bloom taxonomy [4]. Questionnaires may help to check if certain concepts have been understood. Tracing

program execution may support students in understanding code before they have to write their own [9, 11]. *Framed* exercises (e.g. multiple choice questions or filling blanks in code) may provide an easily accessible first step towards programming. But eventually students have to write programs completely on their own – which is usually trained by weekly programming exercises. We denote such exercises as *unstructured* as no scaffolding for the solution is given, but compared to open-ended exercise, still have a clear definition of done. To be of use in a software development team, assessing the programming skills in the latter type of exercises seems most appropriate. However, in comparison the assessment of *framed* exercises is much easier: The spectrum of solutions is limited by a restricted number of proposed answers and the scope of the question is limited to, say, a certain instruction type. In contrast, with *unstructured* exercises many different instruction types have to be combined meaningfully – and different compositions may solve the problem equally well. Assessment becomes even more challenging if, as in our courses, the students may choose freely between different exercises (to account for the heterogeneity of the participants) and have a whole week to solve the exercises (rather than a controlled examination setting), which includes the possibility of helping each other or searching the Internet. So our research question is: *Is it possible to automatically assess the students' individual programming capabilities and distinguish different skills from unstructured coding exercises alone? Can we recognize different groups of students in terms of how well their skills have been developed?* A positive answer may enable us to adapt the course speed and content to the progress of individual students and to automatically recommend exercises that are appropriate for the current level of knowledge, rather than overwhelming students with new concepts. The main contributions of the paper are: (i) a metric for unstructured assignments that captures the performance on exercises (and correlates well with manually graded exams), (ii) a simplified model based on the performance metric which already offers detailed insights into the overall difficulty of individual assignments and students' skills, and finally (iii) a factor model that allows to differentiate the development of skills on different aspects of the programming language (such as *control flow* or *arrays*).

The paper is organized as follows: We review related work in Section 2 and the setting from which we gathered the data for our experimental evaluation in Section 3. Two Matrix Factorization approaches that model how well students have developed different skills are presented in Section 4. This proposal is evaluated on the data from one term in Section 5. Finally, Section 6 concludes the paper.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Koli Calling '24, November 12–17, 2024, Koli, Finland

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1038-4/24/11

<https://doi.org/10.1145/3699538.3699552>

2 Related Work

The research of this work can be assigned to the field of *Knowledge Tracing (KT)*, which deals with the automated tracking of changing student knowledge and was first introduced [1] in 1990 by Anderson et al. [2]. Since then, a variety of approaches were developed. One popular approach is *Bayesian Knowledge Tracing (BKT)* by Corbett and Anderson [7], who first applied it on Lisp programming exercise data. BKT models the probability of having learned a particular skill based on a sequence of interactions with exercises that test for that skill. It comes with many assumptions, e.g., that all exercises depend on mastering a single skill, but allows to directly derive information about a student's skill mastery from the model's parameters.

KT in the field of programming comes with several challenges: First, programming is a complex many-faceted skill, which makes it hard to delimit it into separate skills, also called *Knowledge Components (KCs)*, which is required for many KT approaches. Second, programming exercises usually do not test for just one, but multiple KCs. This requires a mapping of KCs to exercises, which usually requires expert knowledge and manual work. Third, when having programming exercises with multiple KCs, it is challenging to decide which skills a student demonstrated to which degree, which is essential to decide on the mastery of the individual KCs. Unstructured coding exercises usually offer multiple different solution approaches to achieve the same expected results, making it even harder to evaluate the demonstrated skills. This might be the reason why the majority of related research, e.g. [5, 6, 24], relies on data from framed programming exercises [15].

There are many examples of KT research on unstructured coding exercises. Kasurinen and Nikula [12] apply BKT and compare Python code submissions to sample solutions to determine if certain KCs were applied. They model the student population as a homogeneous group, so their model is not able to estimate the individual understanding of different KCs among students. Rivers et al. [19] also rely on unstructured exercises, but apply a KT approach called *Additive Factor Model (AFM)* and focus on analysing the learning curves of KCs, to find out which KCs cause the most difficulties for students. They automatically extract the KCs from the abstract syntax trees of the solutions. A more recent work on unstructured exercises from Shi et al. [20] proposes the use of deep learning for knowledge tracing. As usual for these models, they lack interpretability resulting in an opaque knowledge state representation.

Classical KT approaches like BKT and AFM explicitly model the student's knowledge and learning progress. The related field of *Student Performance Prediction (SPP)* however focuses e.g. on predicting how well students perform in exams or the likeliness that they will graduate. If those predictions are performed by models trained on data about the students' prior performance we could assume that the models encode the students' knowledge in an implicit way. Thai-Nghe et al. propose in [23] the use of recommendation systems, especially *Matrix Factorization (MF)* models for SPP. A common use case for these models is modelling user preference in regard of movies [14], which could be transferred to modelling student performance in programming exercises.

The basic MF model factorizes a sparse matrix $r \in \mathbb{R}^{|U| \times |I|}$, which captures the interaction of users $u \in U$ with items $i \in I$, to a user matrix $p \in \mathbb{R}^{|U| \times f}$ and item matrix $q \in \mathbb{R}^{|I| \times f}$. Both, p and q ,

are fitted in a way that their dot product $\hat{r} = p \cdot q^T$ approximates the known interactions in r . Every user u and item i is represented by a vector p_u resp. q_i in an f -dimensional latent factor space. [14]

Thai-Nghe et al. discuss that these latent factors can implicitly encode information of students or exercises classical KT models like BKT model explicitly, for example the probability to guess the correct answer of an exercise [22]. They also explore different extensions of the basic MF model, e.g. to capture temporal effects [22]. While Thai-Nghe et al. in [22] focus on predicting performance on an exercise level, other authors like Sweeney et al. [21] or Polyzou and Karypis [18] apply MF models to predict next-term grades. These works have in common that they neither explore the model's parameters in order to derive insights about the students and items, nor do they focus on the domain of programming exercises. To the best of our knowledge there is no research yet, which applies MF models for the purpose of KT in the context of unstructured programming.

3 Dataset

This work builds upon a dataset collected during an entry level Java programming course scheduled for the first term. As the course proceeds from week to week, students work on different sets of unstructured programming exercises¹ which deal with the current course material. An exercise consists of a textual description of a set of requirements the solution should fulfil. To take the heterogeneity of the group into account (roughly one third had no experience at all, one third practised a little in school, one third had several months of programming experience), the students could choose (on average) 4-5 out of 7-10 offered exercises every week. This was meant to neither overcharge novices nor bore experienced students, but as a consequence there is not a single exercise that has been worked on by all students.

The students were supported by a plugin [10] for the integrated development environment (IDE), which allows them to access the exercises within the IDE and runs predefined unit tests (each time they save their code) to provide them instant feedback. They had to finish their exercises within one week, but worked on them at their own pace. The tight integration in the IDE allowed us to collect data about their progress without any disruption. Each time the code is evaluated, the results are stored in a server log file, allowing us to track the progress on a specific attempt. Each data point contains attributes characterizing the source code like its McCabe complexity [17], information about the passed and failed unit tests as well as meta-data like a timestamp. After data preprocessing the data set includes 182 thousand log records characterizing 4472 attempts on 100 unique exercises by 149 unique students. During preprocessing we removed attempts with less than three log entries since they offer too little information about the solution generation process, as well as attempts for which more than 75% of the unit tests were passed after less than one minute of work. In such cases the students could have prepared the source code solution either in another IDE or might have copied it from other students.

¹ e.g.: Write a function `mostfreq`, which finds the most frequently occurring value in the passed `int`-array, given that the array contains only positive numbers less than 10000. Yield -1 in case of an empty array.

Upon installation of the IDE plugin it was stated that progress data will be collected and that it will be used for research purposes. The students had the choice of opting out and alternative ways of getting feedback were offered. All students worked on their exercises in the same way as in the terms before (at their own pace, free to use search engines for help, etc). The feedback provided by the plugin was formative in nature and it did not play any role in grading.

For the evaluation we asked the students at the beginning of the course for the number of months of prior programming experience (self-assessed). The results of three programming exams held during the term (carried out in the IDE) as well as of the final course exam (on paper) were available for comparison. All sources were pseudonymized before they were combined and analysed.

4 Approach

If a model is able to predict how well a student performs on a programming exercise, its model parameters need to encode factors or information which characterize the performance of the students. Those factors could originate on the one hand from properties of the programming exercise, e.g. the underlying course topic or the complexity of the exercise. Besides that, temporal factors like the point of time in the term could have an impact. There might be global temporal factors which influence either all students, for example a due date in another course, or individual temporal factors like a drifting amount of motivation. However, the probably most obvious influence factor is the knowledge level of the individual students.

Following this assumption we pursue the idea, that, after training a model to predict the student performance in a first step, we can investigate the model's parameters to gain insights about the students' knowledge states. We investigate the use of MF models which can be trained solely based on a matrix r which reflects the performance of the users $u \in U$ (students) working on the exercise items $i \in I$ (we stick to the identifiers u for users (here: students) and i for items (here: exercises) to be consistent with the literature on recommendation systems). Compared to classical KT approaches, we may not need to define a set of programming KCs beforehand and find a way to decide which of these skills a student demonstrated. Another advantage is that the models' parameters can be categorized into those that characterize students and those that characterize exercises, facilitating the interpretation of the parameters. Furthermore, these models can be extended easily to include temporal parameters to account for the fact that learning is a time-related process. In subsequent sections, we address three major tasks: the model selection (Section 4.1), defining a meaningful measure to express the students' performance on a single exercise (Section 4.2), and finally interpreting the meaning of the derived factors (Section 4.3).

4.1 Modelling

By introducing the time dimension, the matrix of known ratings evolves into a sparse tensor $r \in \mathbb{R}^{|U| \times |I| \times |T|}$ reflecting the performance of the users U (students) on exercise items I at days T in the term. We define the sets $I_u \subseteq I \times T$, $U_i \subseteq U \times T$ and $K_t \subseteq U \times I$

which contain all observations of a specific student u , exercise i or day t , resp.

The basic MF model $\hat{r}_{ui} = p_u \cdot q_i^T$ is often extended by bias parameters $b_u \in \mathbb{R}$ for user u and $b_i \in \mathbb{R}$ for item i to capture systematic tendencies [14]. They enable us to model effects like a specific user u (student) performing generally better than others, or exercise item i being less complex than others. We also add a bias parameter b_t , as introduced in [25] by Xiang and Yang, for each day $t \in T$ of the term to capture global temporal effects. To cover more fine granular time effects, we additionally introduce b_{ut} modelling the bias shift of an individual student on day t . Here we apply an approach introduced by Koren in [13] to model the temporal shifts using spline functions, which are effectively functions of time, which allow to determine the bias shift b_{ut} of each user u on a specific day t of the term. To achieve this, the observed time range of each user u is split up into s_u support points $\{t_1, \dots, t_{s_u}\}$ (in our case one every 20 days – determined by hyperparameter search), which are evenly distributed over the period. For each support point a learnable parameter $b_{t_l}^u$ is introduced whose time-weighted combination results in the spline of a student, with γ controlling its smoothness (set to 0.191 using hyperparameter search):

$$b_{ut} = \frac{\sum_{l=1}^{s_u} \exp^{-\gamma|t-t_l|} b_{t_l}^u}{\sum_{l=1}^{s_u} \exp^{-\gamma|t-t_l|}}$$

Combining the types of parameters introduced, we obtain our Base model which predicts the performance $\hat{r}_{uit}$ of user u (student) on exercise item i at day t as follows (μ : mean of all known values in r):

$$\hat{r}_{uit} = \mu + b_u + b_i + b_t + b_{ut} \quad (Base)$$

This model does not include any latent factors but already provides useful insights as we will see in Section 5.2. Our second model (Factor) extends this model using latent factors p_u and q_i . We prefer *non-negative* factors since they are easier to interpret. The approach implies that the entire model (including the bias parameters) has to be non-negative. We follow the approach of Luo et al. [16] and apply their strategy to ensure non-negativity during training for all the parameters. In addition, the global average μ needs to be removed completely from the model since it would introduce a lower boundary for all predictions. Thus, our second model is as follows:

$$\hat{r}_{uit} = b_u + b_i + b_t + b_{ut} + p_u q_i^T \quad (Factor)$$

Results from a third model, which includes the temporal change of student factors p_u using splines, will not be discussed here, as the amount of the available data did not match the increased number of parameters.

The models are fitted using gradient descent and Tikhonov regularization is applied to all model parameters to mitigate overfitting. Table 1 lists the additive (left) and multiplicative (right) update rules for ordinary and non-negative models to minimize the squared error, since not all of them are available in the referenced literature. Algorithm 1 shows how these update rules can be integrated to fit the model parameters.²

²source code available at: <https://github.com/fpfuetsch/EPSMF>

Table 1: Gradient descent update rules for learnable parameters $\mathcal{P}$ (error: $e_{uit} = r_{uit} - \hat{r}_{uit}$; η : learning rate; λ : regularization factor; j : factor index)

$\mathcal{P}$	ordinary rule ($\mathcal{P} +=$)	non-negative rule ($\mathcal{P} :=$)
b_u	$\eta \sum_{(i,t) \in I_u} e_{uit} - \lambda b_u$	$\frac{\sum_{(i,t) \in I_u} r_{uit}}{\sum_{(i,t) \in I_u} \hat{r}_{uit} + \lambda b_u}$
b_i	$\eta \sum_{(u,t) \in U_i} e_{uit} - \lambda b_i$	$\frac{\sum_{(u,t) \in U_i} r_{uit}}{\sum_{(u,t) \in U_i} \hat{r}_{uit} + \lambda b_i}$
b_t	$\eta \sum_{(u,i) \in K_t} e_{uit} - \lambda b_t$	$\frac{\sum_{(u,i) \in K_t} r_{uit}}{\sum_{(u,i) \in K_t} \hat{r}_{uit} + \lambda b_t}$
p_{uj}	$\eta \sum_{(i,t) \in I_u} e_{uit} q_{ij} - \lambda p_{uj}$	$\frac{\sum_{(i,t) \in I_u} r_{uit} q_{ij}}{\sum_{(i,t) \in I_u} \hat{r}_{uit} q_{ij} + \lambda p_{uj}}$
q_{ij}	$\eta \sum_{(u,t) \in U_i} e_{uit} p_{uj} - \lambda q_{ij}$	$\frac{\sum_{(u,t) \in U_i} r_{uit} p_{uj}}{\sum_{(u,t) \in U_i} \hat{r}_{uit} p_{uj} + \lambda q_{ij}}$
$b_{t_l}^u$	$\eta \sum_{(i,t) \in I_u} e_{uit} \frac{\exp^{-\gamma t-t_l }}{\sum_{l=1}^{s_u} \exp^{-\gamma t-t_l }} - \lambda b_{t_l}^u$	$\frac{\sum_{(i,t) \in I_u} r_{uit} \frac{\exp^{-\gamma t-t_l }}{\sum_{l=1}^{s_u} \exp^{-\gamma t-t_l }}}{\sum_{(i,t) \in I_u} \hat{r}_{uit} \frac{\exp^{-\gamma t-t_l }}{\sum_{l=1}^{s_u} \exp^{-\gamma t-t_l }} + \lambda b_{t_l}^u}$

Algorithm 1 Model fitting (apply corresp. rules from Table 1)**Require:** $U, I, T, U_i, I_u, K_t, r, \lambda, \gamma, s, \text{maxIteration}$, (model dependent: η, f)

```

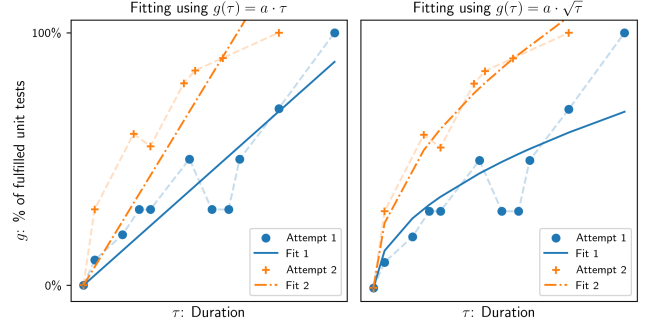
1: initialize model parameters; iteration = 0
2: while iteration < maxIteration do
3:   for all  $u \in U$  do
4:     update  $b_u$ 
5:     for all  $j \in \{0, \dots, f\}$  do
6:       update  $p_{uj}$  {only if Factor model}
7:       for all  $t_l \in \{t_1, \dots, t_{s_u}\}$  do
8:         update  $b_{t_l}^u$ 
9:   for all  $i \in I$  do
10:    update  $b_i$ 
11:    for all  $j \in \{0, \dots, f\}$  do
12:      update  $q_{ij}$  {only if Factor model}
13:   for all  $t \in T$  do
14:     update  $b_t$ 
15:   iteration ++

```

4.2 Performance Metrics

There is a variety of ways to derive a performance metric from our data. A metric could focus on the quality of the final version of the source code a student submitted for a specific exercise. We may take the percentage of passed unit tests or rate the McCabe complexity of the code compared to the complexity of the reference solution. We could also use meta-information such as the total time required to solve the exercise. Or we may exploit that we have a series of data points for each exercise which describes the intermediate steps of the solution finding process. A performance metric which considers not only the final source code version but the entire process of achieving this solution could allow a more granular estimation on the students' knowledge and skills.

Inspired by this idea, we develop a way to derive a single numeric performance value for every exercise attempt. We examine for each attempt the percentage of fulfilled unit tests g in relation to the elapsed time τ a student was working on a specific exercise. This relation $g(\tau)$ can be extracted from our data. Figure 1

**Figure 1: Exemplary approximation of two solution attempts (scatter points) using different single-parameterized functions (left: linear, right: square root)**

depicts this relation for two exemplary attempts. We then fit a single-parameterized function like the linear function $g(\tau) = a \cdot \tau$ or the square root function $g(\tau) = a \cdot \sqrt{\tau}$ to the data. The parameter a which is fitted using a basic least squares method is then used as performance rating r_{uit} of user u (student) on exercise item i at a certain attempt on day t . This means, we can compute a **student progress (SP)** metric SP_{at} which assigns our example attempt #1 (Fig. 1) a higher value than attempt #2, based on the higher gradient value a for attempt #1, indicating a better performance. In comparison, if we would only focus on the final portion of passed unit tests, since both attempts reach 100% at some point, both would receive the same metric value, ignoring faster progress in attempt #2. To maintain the comparability of the SP metric among exercises with different complexity (and therefore different durations), we normalize the time axis beforehand based on the median duration students effectively worked on an exercise. We estimate the effective working duration by assuming students paused their work, when they did not evaluate their code for 2 hours using the IDE plugin. This is necessary to cope with the varying degree of previous knowledge: While some students may solve the tasks in less than 2 hours, unexperienced students often start on one day, struggle at some point, and continue working on subsequent days. A break of at least 2 hours turned out to be a good indicator for this type of task processing.

We prepare a total of eight different performance metrics for our dataset for further investigation which include the percentage of **passed tests**, the percentage of passed tests per hour (**tests/hour**) and a metric which honours low **McCabe** complexity weighted by the portion of passed unit tests. Additionally, we compute five SP metrics using different single-parameterized functions: $a\tau$, $a\tau^2$, $a\sqrt{\tau}$, $a \log \tau$ and $a + \log \tau$. In order to prepare the SP metric values to be used as input data for our models we apply a log transformation and 99.9% quantile clipping to counteract skewness and outliers of the metric distribution, as well as rescaling to $[0,1]$ to have a

standardized value range. By doing so we achieve a distribution of the metric value which better approximates a normal distribution – illustrated exemplarily in Figure 2 for the metric $SP_{a\sqrt{\tau}}$.

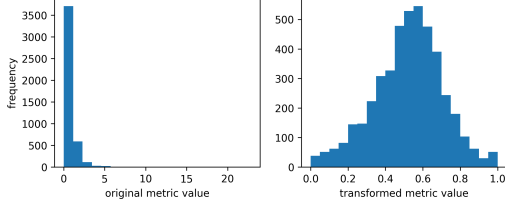


Figure 2: Histograms (20 bins) for the original (left) and transformed (right) metric values of $SP_{a\sqrt{\tau}}$ of all 4472 exercise attempts.

4.3 Factor Meaning

It is necessary to understand the latent factor space spanned by the MF models in order to interpret the individual factor values for students and exercises. This is a complex task because a single factor may encode multiple aspects.

There are approaches like [8] which try to interpret the factors by predicting them using shadow models based on known metadata features about the entities (exercises in our case). The metadata features which are useful to predict the latent factor are then expected to be associated with their meaning. Compared to the movie domain of [8] we lack extensive metadata about our exercises. Instead, we derive it from the exercises themselves: We assume that the exercise factors q_i encode KCs, especially course topics, which are relevant to solve the exercise item i and the related student factors p_u encode how well the user u (student) masters these KCs. The according programming course can be divided into four high level topics to each of which we assign a set of relevant language constructs: usage of **variables of primitive types (Basics)**, **control flow (CtrlFlow)** including conditional statements and loops, **arrays & functions (ArrFunc)** as well as object-oriented constructs for **classes & methods (ClassMeth)**. We apply an abstract syntax tree parser on the reference solution for each programming exercise to automatically extract the number of language constructs falling into these four categories, serving as our exercise metadata.

We then try to determine the latent factor meaning by correlating the exercise factors q_i with the number of corresponding language constructs of these four groups. However, as our dataset is rather small, we may be forced to an even simpler approach: We may *pre-define* the meaning of the exercise factors q_i with the given percentages of contained language constructs. The model then effectively learns only the student factors p_u (omit line 12 of Alg. 1). Table 2 gives examples for the percentages of the four categories of language constructs. The exercise *Bit5*, which can be solved using only basic usage of primitive variables, will then be represented in the model using the four factors $q_{Bit5} = (1, 0, 0, 0)^T$. In contrast, exercise *EggPack* requires language constructs from each group and is represented by $q_{EggPack} = (0.16, 0.26, 0.31, 0.27)^T$.

Table 2: Extract of the pre-defined exercise factors

exercise	Basics	CtrlFlow	ArrFunc	ClassMeth
Bit5	100%	0%	0%	0%
CapitalCount	43%	57%	0%	0%
ColorBall	23%	33%	44%	0%
DistanceUnit	30%	0%	0%	70 %
EggPack	16%	26%	31%	27%

5 Experimental Evaluation

Recalling the research question, we want to know if the programming capabilities of the students can be assessed meaningfully from unstructured exercises by the proposed approach. Our focus in this work does not lie on comparisons with other approaches regarding predictive performance but rather on its feasibility. As already outlined in Section 2, a direct comparison with classical knowledge tracing methods is not possible, because they require a more structured type of data (e.g. questionnaires) rather than data describing source code evolution over time.

All model trainings and estimations took less than a minute on consumer hardware. As already mentioned in Section 4.1, hyperparameters were optimized systematically using *hyperopt* [3] in terms of the root mean square error (a common metric in the context of MF) while applying 5-fold cross-validation. In addition to those parameters already mentioned, the optimization also included individual regularization parameters λ for each type of learnable parameter as well as the learning rate η of the Base model.

5.1 Performance Metric

First, we have to investigate which of the proposed measures (cf. Section 4.2) captures the students’ performance best. Three programming exams (mini exams) have been conducted during the term, which were graded manually and serve as a reference. As these exams were relevant for the final grade, the students had a strict time limit for solving the exercises and the use of any aids was prohibited. However, these circumstances are very different from the ordinary self-paced exercises. Therefore, for each mini exam we identified three standard exercises which have been solved shortly before the exam and cover the same topics. As it is this type of exercise from which we want to learn about a student’s programming skills, the correlation of a student’s average metric for those three exercises with the grades of the mini exams is considered much more relevant. Table 3 shows the Pearson correlation of the manual grades grouped by weeks 5, 8, and 11. Within each group, the left column shows the correlation with the metrics obtained for the exam itself, and the right column the correlation with the average metric obtained for the three selected ordinary exercises. In each column, the highest three correlations are printed in bold face. Only the $SP_{a\sqrt{\tau}}$ and $SP_{a\log \tau}$ metric are bold in all six columns. For all further experiments we settled on $SP_{a\sqrt{\tau}}$ as it correlates somewhat stronger.

The passed tests alone also correlate well with the manually derived grades of the mini exams. But this correlation is misleading for the aforementioned reasons: It drops for the ordinary exercises (especially in week 8 and 11) by a larger margin than the selected

Table 3: Pearson correlation between three manually determined grades and proposed performance metrics. Grouped by mini exam: metric of the exam exercise attempt itself (left) and average metric of three highly related ordinary exercises (right). Top three correlations in bold.

metric	exercises week 5		exercises week 8		exercises week 11	
	exam #1	ordinary (n=107)	exam #2	ordinary (n=94)	exam #3	ordinary (n=90)
passed tests	0.478	0.504	0.813	0.425	0.675	0.390
tests/hour	0.303	0.639	0.795	0.458	0.496	0.413
McCabe	0.058	0.023	0.708	0.122	0.358	0.266
$SP_{a\tau}$	0.307	0.644	0.803	0.515	0.631	0.420
$SP_{a\tau^2}$	0.145	0.611	0.749	0.479	0.468	0.343
$SP_{a\sqrt{\tau}}$	0.419	0.642	0.811	0.551	0.706	0.472
$SP_{a\log \tau}$	0.330	0.649	0.808	0.505	0.649	0.431
$SP_{a+\log \tau}$	0.266	0.563	0.644	0.496	0.543	0.439

$SP_{a\sqrt{\tau}}$ metric (compare “exam” and “ordinary” column). One reason may be the number and type of unit tests (more fine-grained tests were prepared for the exams). Another reason is that for weekly exercises (in contrast to exams) students eventually come up with a solution that works – either because they invested much more time or they asked other students for help. If all students eventually pass all tests, the time spent on an exercise and the speed of progress becomes a crucial factor of the assessment to distinguish skill levels – which explains the higher correlations of the SP metrics. And as already mentioned, the vast majority of exercises that were solved by the students are of the ordinary type, so the correlation in the “ordinary” columns of Table 3 is more important than the correlation in the “exam” columns (only 3 exams but 100 ordinary exercises).

5.2 Bias Parameters

Once the submissions have been rated using the selected metric (Section 5.1), the models of Section 4.1 can be fitted. First we consider the model Base that consists of bias terms only.

Exercise difficulty: The bias b_i for exercise item i can be interpreted as the difficulty of the exercise. Exercises are perceived as difficult for many reasons: some exercises may require more computational thinking, some assume some mathematical skills before programming the solution, some exercises require careful reading, etc. We thus do not expect a strong correlation of the exercise bias b_i with any single property. It thus appears reasonable that the McCabe complexity (as obtained from a reference solution), addressing code complexity only, correlates with b_i only slightly $\rho = -0.22$ (significant, p-value 0.0015). Furthermore, the exercise difficulty b_i may occasionally be biased by the group of students who has selected it: An introductory exercise may be judged as difficult by the group of novices – which remains undisputed as none of the advanced students ever selected it. From a supervisors perspective, however, the automatically derived bias b_i corresponds well to a *general difficulty* of the respective exercise.

Student skill: The user bias b_u may capture the programming capabilities for student u . To evaluate the meaningfulness of b_u we use the final grade obtained in the course: A Pearson correlation coefficient of $\rho = 0.6189$ (p-value $3e-19$) indicates that this parameter alone captures the programming capabilities quite well (assuming the final exam does so also). Such a high correlation is remarkable,

Table 4: Definition of groups of students, based on a questionnaire before and the final exam after the course.

group	abr.	experience	final exam
experienced-successful	ES	$> q_{0.80}$	$> q_{0.75}$
experienced-unsuccessful	EU	$> q_{0.80}$	$< q_{0.50}$
unexperienced-successful	US	$< q_{0.40}$	$> q_{0.50}$
unexperienced-unsuccessful	UU	$< q_{0.40}$	$< q_{0.25}$
resigning	RE	underperf. in mini exams, not taking the final exam	

because the final exam was written on paper (rather than in the IDE) and thus consisted of somewhat different types of exercises.

Daily bias: The time bias b_t indicates if students performed generally worse or better on specific days t . We observed a tendency of higher productivity towards the lab days, where new exercises were issued and the students benefited from the availability of supervisors during the lab to clarify difficulties straight away. Various days in between exhibited a negative bias, which is probably due to the fact that less experienced students still worked on the exercises that were already solved by the experienced students, which leads to an overall negative bias on such days.

Skill evolution: Finally, we consider the change in the student skill (b_u) over time as captured by the splines b_{ut} . We assess the trend in a student’s skill by subtracting the spline value at the end of the term from the spline value at the beginning of the term. A positive value indicates that the student bias has increased and thus the student has improved. Figure 3 (top) shows the boxplot of b_u (blue) together with the *trend* (red) for the group of students that did not write (na) / have failed (false) / have passed (true) the final exam. The group of students who have passed the final exam are clearly separated from the other groups in the b_u value. But the trend boxplots look quite similar (median close to 0, small lower quartile, somewhat larger upper quartile). We thus take a closer look on certain subgroups of students, as defined in Table 4. The group of, e.g., experienced and successful students (ES) replied to “programming experience in months” (in the initial questionnaire) above the 0.8-quantile of all replies and achieved a total number of points in the final exam above the 0.75-quantile. (Note that for the EU group “unsuccessful” does not mean that they did not pass the exam, only that their grade was below the 0.5-quantile of all grades.) The average group size is 13.4 students. Figure 3 (bottom) shows the student bias (blue) and trend (red) for these groups. It is interesting to observe that for all unexperienced students (US, UU but also RE) the upper quartile of the trend values stretches further into the positive range, which indicates that they have improved by a larger margin than the experienced students. It is surprising to see that the group of unexperienced but successful students (US) has achieved a skill bias as high as the group of experienced but unsuccessful students (EU). However, the trend boxplots differ, with US students improving by a greater margin than the EU students, who seem to be resting on their laurels rather than improving their skills.

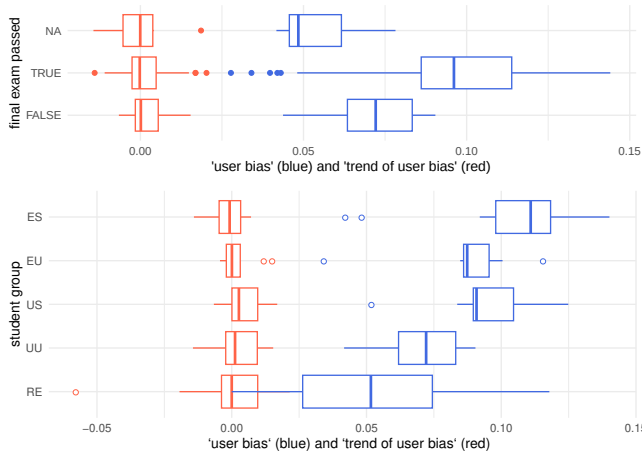


Figure 3: Student bias and change/trend of student bias.

5.3 Factors

Next we consider the Factor model. The interpretation of the automatically derived factors turned out to be difficult, on the one hand because the reasons why an exercise may be perceived as difficult are manifold (see previous section) and on the other hand because the gradient descent converged to different factors p_u and q_i in consecutive runs. Correlations to the exercise properties (Basics, CtrlFlow, ArrFunc, ClassMeth) were rather low. Lacking further metadata to correlate the factors with, it was difficult to grasp their meaning. We thus settled for pre-defined exercise factors q_i , automatically derived from the reference solutions (cf. Section 4.3), which greatly stabilized model fitting.

The factors correlate only slightly among each other (lowest/highest Pearson correlation of 0.03/0.31) and sometimes well with the student bias (lowest/highest Pearson correlation of 0.37/0.6). A linear regression model to predict the final exam points that uses the student bias b_u alone achieves $R^2 = 0.359$, but using the four factors improves the model to $R^2 = 0.599$. Using the linear combination of factors from the regression model, the correlation to the final exam increases to 0.757 (versus 0.62 for b_u alone). The four factors thus characterize the students programming capabilities much better than the student bias parameter alone.

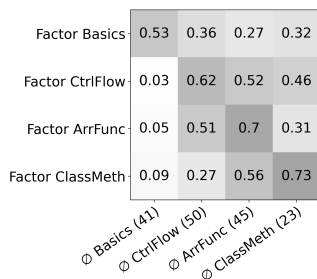


Figure 4: Pearson correlation of factors to different groups of exercises (groups by column, groups size in parentheses, all p-values < 0.05).

Figure 4 shows how the factors correlate with the performance metric from Section 5.1 for different groups of exercises: Depending on the dominating language constructs (cf. Table 2), each exercise was assigned to one group (e.g. Bit5 would be assigned to group Basics, EggPack to ArrFunc). For each student, we determine the average performance within each group and correlate it with the resp. factor. The correlations on the diagonal are the highest, which means that for the group of exercises that require knowledge on, say, classes and methods, the factor ClassMeth captured the student's performance best. This shows how factors allow us to differentiate between different programming skills of the student.

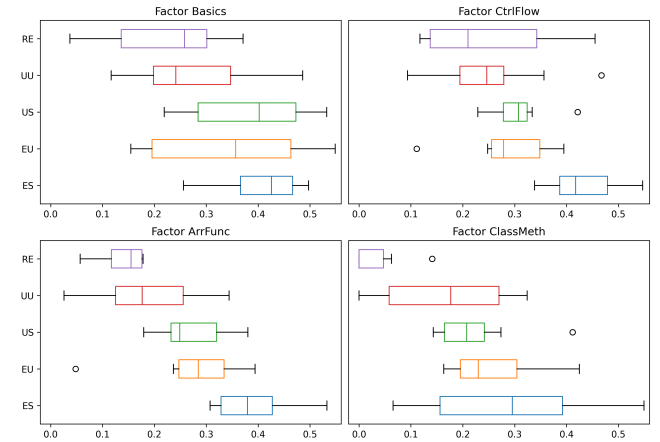


Figure 5: Boxplots of factors per student group.

Figure 5 shows boxplots of all factors by student group. For the more advanced factors (ArrFunc and ClassMeth) the median values are strictly ordered by student group (as expected). For the first two factors, however, the median of unexperienced but successful students (US) surprisingly surpasses the median of experienced but unsuccessful students (EU) – students from the EU group may have underestimated the exercises or overestimated their own skills. It is also interesting to observe that the boxplots of the US group are quite compact, which may indicate that the course-driven knowledge build-up is homogeneous – they follow and understand any new content consistently and do not struggle with misconceptions that were established earlier. In contrast, the boxplot of the UU group widens with advanced topics (CtrlFlow to ClassMeth), indicating that their level of understanding diverges as the course evolves. From the boxplots of the ES group one may conjecture that they have experience in the area of control flow, arrays and functions, but not so much experience with classes and methods, as the compact boxplots widen considerably for the latter factor.

Figure 6 indicates how the four factors relate for the Basics/CtrlFlow (left) and ArrFunc/ClassMeth case (right) for the four groups UU, US, EU, and ES. In the left figure, groups ES and UU are well separated, EU and US may represent a transitional status and are hardly distinguishable in terms of factors. In the right figure, the ES group is the most compact and homogeneous group. The EU group is comparable to ES in terms of the ClassMeth factor, but has a much higher variance in the ArrFunc factor. For the US group both factors vary greatly, the group is inhomogeneous.

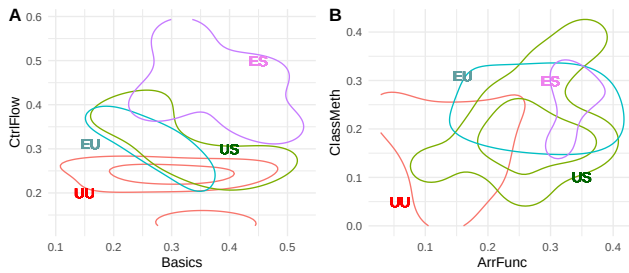


Figure 6: Groupwise density plot for factor combinations.

When comparing the group of students who have passed the final exam versus the group of students who did not, the mean of the Basics factor does not differ significantly, but all other factors do on a 95% confidence level (p-values: CtrlFlow: $2.7e-5$, ArrFunc: $9.7e-7$, ClassMeth: 0.023). This allows us to provide individual feedback to each student (regarding the three aspects CtrlFlow, ArrFunc, ClassMeth) whether the required skills have already been sufficiently developed to likely pass the exam (or suggest to practice related exercises to improve further). By monitoring the development of these factors over the term, we may be able to identify students who need additional support at an early stage.

6 Conclusions and Outlook

We investigated the research question whether it is possible to automatically assess the students' individual programming capabilities from unstructured exercises. Our study demonstrates that this is possible: Without distracting students from their work on programming exercises in the IDE, the temporal evolution of passed tests and the properties of sample solutions were sufficient to derive a Matrix Factorization model, which provides insights into the (perceived) difficulty of exercises, the programming skills of students and the individual degree of improvement. After settling for pre-defining the factor meaning, the method also allowed us to differentiate the skill levels with respect to different programming language constructs, confirming the second aspect of our research question. The high correlation with midterm and final exams results validate the overall meaningfulness of the obtained model parameters.

In contrast to common approaches for assessing programming skills, which usually require a well-prepared and aligned set of *framed* questions, the main advantages of the proposed approach are that we (i) employ “real coding exercises” and observe students while they solve them in a “natural setting” (IDE), and (ii) we do not need to prepare or annotate these exercises in advance, but extract the necessary skills automatically from the language constructs contained in the reference solution.

With increasing course sizes (and limited time) it is close to impossible for lab supervisors to track the individual student's performance by personal observations alone. The purpose of this work was to evaluate the usefulness of the models' parameter in terms of student assessment. In the future, insights gained from such models may help lab supervisors to make the best use of limited contact time. For instance, the trend information may be

used in lab situations (by lab supervisors) to quickly assess the situation: Is the candidate improving and a discussion can focus on the details of their solution - or is the candidate struggling and needs encouragement and help with basic concepts. The regression model obtained from the factors, given its high correlation with final exam scores, may be used to provide a meaningful performance feedback to each student individually as part of an open learner model. The automatically derived exercise bias corresponded well to the difficulty of the exercise and can be displayed alongside the exercise to help the student select the next exercise to gain further practice without overwhelming them. Finally, the matrix $\hat{r}$ of predicted student performance on our lab exercises can be used to automatically recommend exercises to students that match their current level of understanding.

References

- [1] Ghodai Abdelrahman, Qing Wang, and Bernardo Nunes. 2023. Knowledge Tracing: A Survey. *Comput. Surveys* 55, 11 (Nov. 2023), 1–37. <https://doi.org/10.1145/3569576>
- [2] John R. Anderson, C. Franklin Boyle, Albert T. Corbett, and Matthew W. Lewis. 1990. Cognitive modeling and intelligent tutoring. *Artificial Intelligence* 42, 1 (Feb. 1990), 7–49. [https://doi.org/10.1016/0004-3702\(90\)90093-F](https://doi.org/10.1016/0004-3702(90)90093-F)
- [3] James Bergstra, Daniel Yamins, and David Cox. 2013. Making a Science of Model Search: Hyperparameter Optimization in Hundreds of Dimensions for Vision Architectures. In *Proceedings of the 30th International Conference on Machine Learning*. PMLR, 115–123. <http://proceedings.mlr.press/v28/bergstra13.pdf> ISSN: 1938-7228.
- [4] B.S. Bloom and D.R. Krathwohl. 1956. *Taxonomy of Educational Objectives: The Classification of Educational Goals*. Number 1. Longmans, Green.
- [5] Kuo-En Chang, Bea-Chu Chiao, Sei-Wang Chen, and Rong-Shue Hsiao. 2000. A programming learning system for beginners-a completion strategy approach. *IEEE Transactions on Education* 43, 2 (May 2000), 211–220. <https://doi.org/10.1109/13.848075>
- [6] Konstantina Chrysafiadi and Maria Virvou. 2015. Fuzzy Logic for Adaptive Instruction in an E-learning Environment for Computer Programming. *IEEE Transactions on Fuzzy Systems* 23, 1 (Feb. 2015), 164–177. <https://doi.org/10.1109/TFUZZ.2014.2310242>
- [7] Albert T. Corbett and John R. Anderson. 1994. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction* 4, 4 (Dec. 1994), 253–278. <https://doi.org/10.1007/BF01099821>
- [8] Anupam Datta, Sophia Kovaleva, Piotr Mardziel, and Shayak Sen. 2018. Latent Factor Interpretations for Collaborative Filtering. <https://doi.org/10.48550/arXiv.1711.10816> arXiv:1711.10816 [cs].
- [9] Frank Höppner. 2019. Measuring Instruction Comprehension by Mining Memory Traces for Early Formative Feedback in Java Courses. In *Proceedings of the 2019 ACM Conference on Innovation and Technology in Computer Science Education (Aberdeen, Scotland UK) (ITI/CSE '19)*. Association for Computing Machinery, New York, NY, USA, 105–111. <https://doi.org/10.1145/3304221.3325529>
- [10] Frank Höppner. 2020. Taking benefit from fellow students code without copying off - making better use of students collective work. In *DELFI 2020 - Die 18. Fachtagung Bildungstechnologien der Gesellschaft für Informatik e.V., online, 14.-18. September 2020 (LNI, Vol. P-308)*, Raphael Zender, Dirk Ifenthaler, Thiemo Leonhardt, and Clara Schumacher (Eds.). Gesellschaft für Informatik e.V., 217–222. <https://dl.gi.de/handle/20.500.12116/34163>
- [11] Maximilian Jahnke and Frank Höppner. 2022. Is there Method in Your Mistakes? Capturing Error Contexts by Graph Mining for Targeted Feedback. In *Proceedings of the 15th International Conference on Educational Data Mining*. International Educational Data Mining Society, 422–429. <https://doi.org/10.5281/zenodo.6853193>
- [12] Jussi Kasurinen and Uolevi Nikula. 2009. Estimating programming knowledge with Bayesian knowledge tracing. *ACM SIGCSE Bulletin* 41, 3 (Aug. 2009), 313–317. <https://doi.org/10.1145/1595496.1562972>
- [13] Yehuda Koren. 2009. Collaborative filtering with temporal dynamics. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining (KDD '09)*. Association for Computing Machinery, New York, NY, USA, 447–456. <https://doi.org/10.1145/1557019.1557072>
- [14] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix Factorization Techniques for Recommender Systems. *Computer* 42, 8 (Aug. 2009), 30–37. <https://doi.org/10.1109/MC.2009.263>
- [15] Philip I.S. Lei and Antonio Jose Mendes. 2021. A systematic literature review on knowledge tracing in learning programming. In *2021 IEEE Frontiers in Education Conference (FIE)*. IEEE, Lincoln, NE, USA, 1–7. <https://doi.org/10.1109/FIE49875.2021.9637323>

- [16] Xin Luo, Mengchu Zhou, Yunni Xia, and Qingsheng Zhu. 2014. An Efficient Non-Negative Matrix-Factorization-Based Approach to Collaborative Filtering for Recommender Systems. *IEEE Transactions on Industrial Informatics* 10, 2 (May 2014), 1273–1284. <https://doi.org/10.1109/TII.2014.2308433>
- [17] T.J. McCabe. 1976. A Complexity Measure. *IEEE Transactions on Software Engineering* SE-2, 4 (Dec. 1976), 308–320. <https://doi.org/10.1109/TSE.1976.233837>
- [18] Agoritsa Polyzou and George Karypis. 2016. Grade prediction with models specific to students and courses. *International Journal of Data Science and Analytics* 2, 3 (Dec. 2016), 159–171. <https://doi.org/10.1007/s41060-016-0024-z>
- [19] Kelly Rivers, Erik Harpstead, and Ken Koedinger. 2016. Learning Curve Analysis for Programming: Which Concepts do Students Struggle With?. In *Proceedings of the 2016 ACM Conference on International Computing Education Research*. ACM, Melbourne VIC Australia, 143–151. <https://doi.org/10.1145/2960310.2960333>
- [20] Yang Shi, Min Chi, Tiffany Barnes, and Thomas Price. 2022. Code-DKT: A Code-based Knowledge Tracing Model for Programming Tasks. In *Proceedings of the 15th International Conference on Educational Data Mining*. <https://doi.org/10.5281/zenodo.6853105>
- [21] Mack Sweeney, Jaime Lester, and Huzefa Rangwala. 2015. Next-term student grade prediction. In *2015 IEEE International Conference on Big Data (Big Data)*. IEEE, Santa Clara, CA, USA, 970–975. <https://doi.org/10.1109/BigData.2015.7363847>
- [22] Nguyen Thai-Nghe, Lucas Drumond, Tomas Horvath, Artus Krohn-Grimberghe, Alexandros Nanopoulos, and Lars Schmidt-Thieme. 2011. Factorization Techniques for Predicting Student Performance. *Educational Recommender Systems and Technologies: Practices and Challenges* (Jan. 2011). <https://doi.org/10.4018/978-1-61350-489-5.ch006>
- [23] Nguyen Thai-Nghe, Lucas Drumond, Artus Krohn-Grimberghe, and Lars Schmidt-Thieme. 2010. Recommender system for predicting student performance. *Procedia Computer Science* 1, 2 (2010), 2811–2819. <https://doi.org/10.1016/j.procs.2010.08.006>
- [24] Shanshan Wang, Yong Han, Wenjun Wu, and Zhenghui Hu. 2017. Modeling student learning outcomes in studying programming language course. In *2017 Seventh International Conference on Information Science and Technology (ICIST)*. 263–270. <https://doi.org/10.1109/ICIST.2017.7926768>
- [25] Liang Xiang and Qing Yang. 2009. Time-Dependent Models in Collaborative Filtering Based Recommender System. In *2009 IEEE/WIC/ACM International Joint Conference on Web Intelligence and Intelligent Agent Technology*, Vol. 1. 450–457. <https://doi.org/10.1109/WI-IAT.2009.78>